VINUNIVERSITY

LLMOps và quản lý phiên bản prompt

AICB-P2T2 · Ngày 22 · Chương 5 — Vận hành

Giảng viên Nguyễn Hải Dương

VinUniversity · Phase 2 · Track 2 · Tuần 5

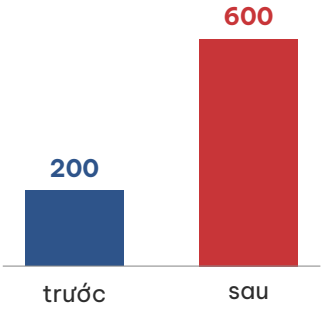
HÃY SUY NGHĨ...

"Một nhóm sửa một dòng trong chỉ dẫn hệ thống (system prompt) để câu trả lời chi tiết hơn. Sau đó độ trễ (latency) và chi phí cùng tăng, và không có bản ghi nào cho biết thay đổi nào gây ra điều đó. Hệ thống cần ghi lại những gì để truy ngược được nguyên nhân?"

Giữ câu hỏi này trong đầu khi học bài hôm nay

Kịch bản: một dòng prompt làm đổi chi phí

- Thay đổi: thêm yêu cầu "trả lời chi tiết hơn" vào prompt (câu lệnh đầu vào gửi cho mô hình).
- Độ dài đầu ra tăng từ 200 lên 600 token (đơn vị văn bản nhỏ nhất mà mô hình xử lý).
- Độ trễ tăng khoảng 3 lần vì mô hình sinh token tuần tự.
- Chi phí tăng khoảng 200% vì giá đầu ra tính theo token.
- Không yêu cầu nào gắn với một phiên bản prompt, nên không so sánh được trước và sau thay đổi.



token đầu ra mỗi câu trả lời

Số liệu giả định để minh hoạ cơ chế, không phải đo đạc thực tế.

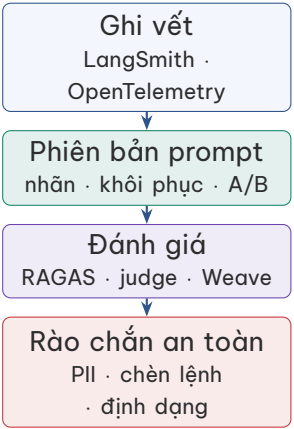
Nội dung bài học

1. LLMOps và MLOps truyền thống
2. Ghi vết và quan sát
3. Quản lý phiên bản prompt
4. Đánh giá mô hình ngôn ngữ lớn
5. W&B Weave và công cụ đánh giá
6. Rào chắn an toàn và giám sát
7. Trình diễn và thực hành

Mục tiêu bài học

Sau bài học này, học viên có thể:

1. Ghi vết thực thi (trace) của ứng dụng mô hình ngôn ngữ lớn (LLM) bằng LangSmith và chuẩn OpenTelemetry, đọc cây vết để tìm điểm nghẽn
2. Quản lý phiên bản prompt: bản ghi thay đổi, nhãn phiên bản, kéo và đẩy, khôi phục phiên bản trước (rollback)
3. Thiết lập đánh giá (evaluation) có hệ thống: RAGAS, LLM-as-judge (dùng mô hình ngôn ngữ để chấm điểm đầu ra), W&B Weave
4. Áp rào chắn an toàn (guardrail) ở đầu vào và đầu ra theo OWASP LLM Top 10 2025



LLMOps và MLOps truyền thống

Vì sao vận hành mô hình ngôn ngữ lớn khác với vận hành mô hình học máy

01

MLOps và LLMOps: các khác biệt chính

	MLOps truyền thống	LLMOps
Ghi nhận	siêu tham số (hyperparameter), chỉ số (metric) huấn luyện và đánh giá	vết từng lời gọi LLM, token và chi phí mỗi yêu cầu
Đầu ra	xác định, tái lập được	không xác định, chất lượng mang tính chủ quan
Phiên bản hoá	trọng số (weight), tập dữ liệu (dataset), mã nguồn	thêm prompt, chỉ dẫn hệ thống, tham số, lược đồ đầu ra
Đánh giá	accuracy, F1, AUC-ROC	faithfulness, mức liên quan, tỉ lệ ảo giác (hallucination)
Mô hình chi phí	giờ tính toán (GPU)	token mỗi tác vụ
An toàn	công bằng, thiên kiến của mô hình	chèn lệnh, vượt quyền, rò rỉ dữ liệu cá nhân
Công cụ tiêu biểu	MLflow, W&B, DVC	LangSmith, Langfuse, Weave, RAGAS, Guardrails AI

Nguồn: tổng hợp của giảng viên từ bản trình bày cũ và tài liệu các công cụ

Sáu thách thức riêng của LLM

Không xác định

Cùng một đầu vào cho đầu ra khác nhau ở mỗi lần chạy. Lỗi khó tái hiện, cần lưu cả đầu vào lẫn đầu ra.

Nhạy với prompt

Đổi một từ có thể đổi hành vi. Prompt cần phiên bản hoá và rà soát như mã nguồn.

Chất lượng chủ quan

Thường không có đáp án chuẩn (ground truth). Cần LLM-as-judge hoặc người đánh giá.

Chi phí theo token

Chi phí tính theo token chứ không theo thời gian tính toán. Prompt dài hơn thì tốn nhiều hơn.

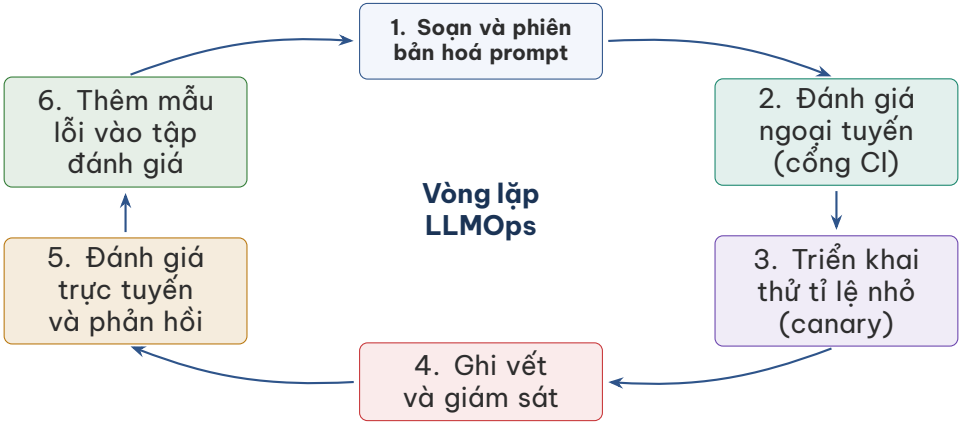
Rủi ro an toàn

Chèn lệnh, vượt giới hạn, rò rỉ dữ liệu cá nhân: không có tương đương trong MLOps truyền thống.

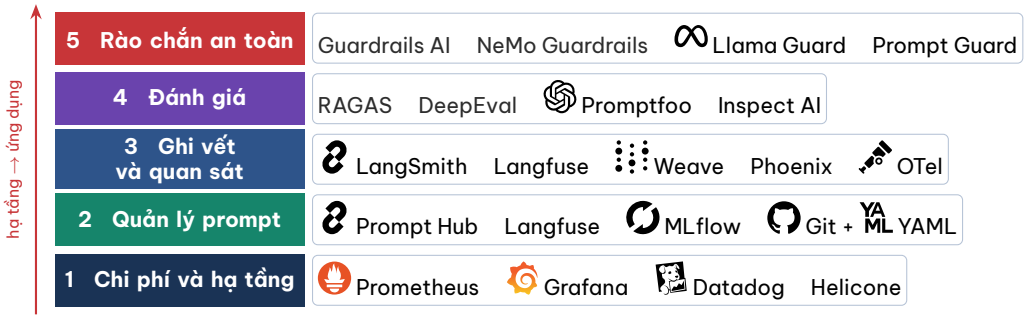
Ảo giác

Mô hình có thể tự sinh thông tin sai. Tỉ lệ ảo giác cần được theo dõi liên tục.

Vòng lặp LLMOps

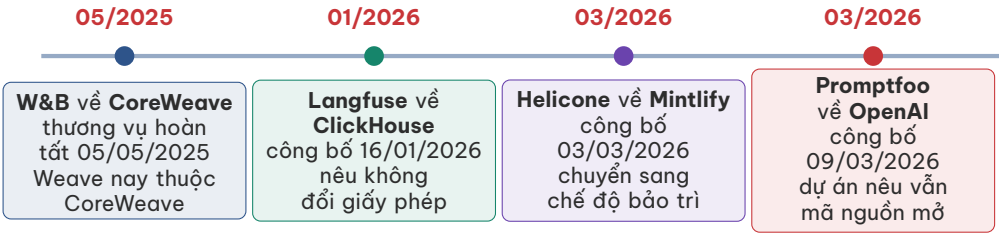


Bản đồ bộ công cụ (stack) LLMOps 2026



Mỗi lớp có thể thay công cụ độc lập nếu dữ liệu đi qua định dạng mở (OpenTelemetry, YAML, JSON).

Thị trường hợp nhất: bốn thương vụ trong 12 tháng



Hệ quả thiết kế: giữ dữ liệu vết ở định dạng OpenTelemetry và prompt ở tệp văn bản, để đổi nhà cung cấp mà không viết lại ứng dụng.

Nguồn: CoreWeave blog; Langfuse GitHub discussion 11593; Helicone blog; README của Promptfoo (chưa đối chiếu trang gốc của OpenAI)

Ghi vết và quan sát

02

LangSmith, OpenTelemetry và cách đọc một vết thực thi

Ghi vết đã phổ biến, đánh giá thì chưa

89%
tổ chức có một dạng quan sát cho agent

62%
có ghi vết chi tiết từng bước và lời gọi công cụ (tool call)

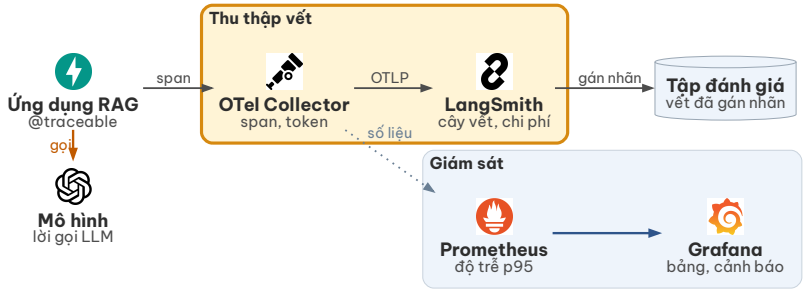
94%
có quan sát, trong nhóm agent đã chạy ở môi trường vận hành

71,5%
có ghi vết đầy đủ, cùng nhóm đó

Agent là hệ thống AI tự chọn hành động và công cụ để hoàn thành mục tiêu. Mẫu khảo sát gồm 1.340 phản hồi, thu từ 18/11 đến 02/12/2025; người trả lời là cộng đồng của chính LangChain nên có thiên lệch chọn mẫu.

Nguồn: LangChain, State of Agent Engineering (langchain.com/state-of-agent-engineering)

Kiến trúc quan sát cho ứng dụng LLM



Ứng dụng phát đoạn (span) qua OpenTelemetry; bộ thu chuyển vết tới LangSmith (cây vết, chi phí, gán nhãn thành tập đánh giá) và số liệu tổng hợp tới Prometheus và Grafana (độ trễ p95, cảnh báo).

Nguồn: sơ đồ do giảng viên vẽ (assets/diagrams/llmops-observability.py); logo theo assets/logos/ASSETS.md

LangSmith: bật ghi vết và gắn `@traceable`

```python
# env: LANGSMITH_TRACING=true, LANGSMITH_API_KEY (secret),
#      LANGSMITH_PROJECT=day22
from langsmith import traceable
from langsmith.wrappers import wrap_openai
import openai

client = wrap_openai(openai.Client())  # each LLM call -> child run

@traceable(run_type="chain", name="rag_pipeline")
def rag_pipeline(question: str) -> str:
    docs = retriever.invoke(question)
    context = format_docs(docs)
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user",
                   "content": f"Context: {context}\nQ: {question}"}])
    return r.choices[0].message.content
```

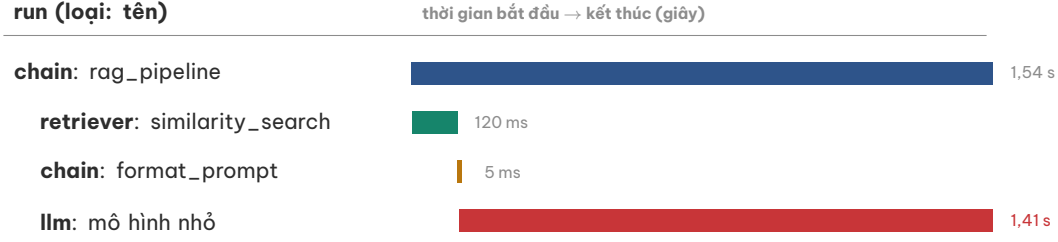
Thay đổi so với bản cũ

- Tài liệu hiện dùng `LANGSMITH_*`; SDK vẫn nhận `LANGCHAIN_TRACING_V2` như biến cũ.
- Các loại run: `chain`, `llm`, `tool`, `retriever`.

Lọc vết để tìm lỗi

- chỉ các vết lỗi
- độ trễ lớn hơn 2 giây
- chi phí mỗi lời gọi vượt ngưỡng
- theo mô hình, nhãn, khoảng ngày

Đọc một vết: cây run và biểu đồ thác nước

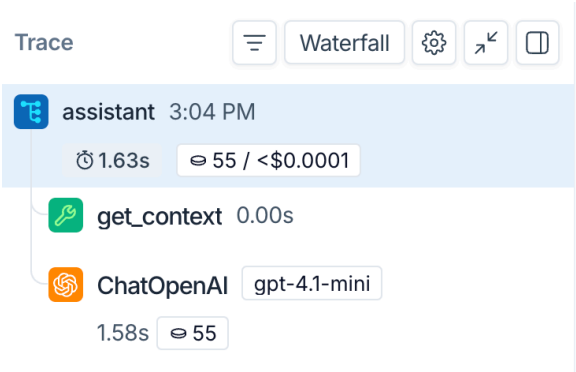


tokens vào: 542
tokens ra: 198
`finish_reason`: stop
độ trễ lời gọi mô hình: 1,41 s

Điểm nghẽn là lời gọi mô hình: 92% thời gian.
Cách giảm: rút ngắn đầu ra, hoặc phát dòng (streaming).

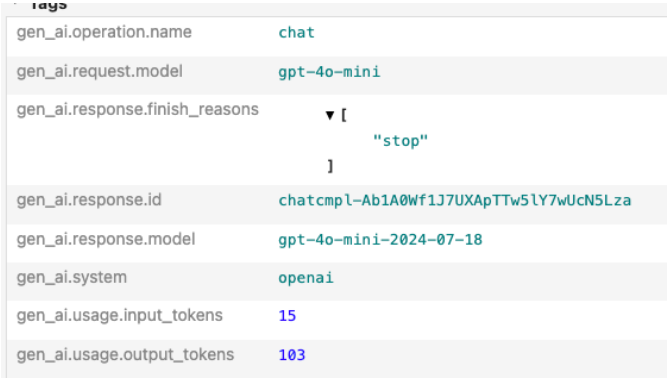
Vết minh hoạ; số liệu dựng để cho thấy cách đọc, không phải đo từ hệ thống thật.

Một vết thật trong LangSmith



1. Cây run: run gốc `assistant`, run con `get_context` (công cụ) và lời gọi mô hình `ChatOpenAI`. Mỗi nút hiển thị thời gian, số token và chi phí.
2. Run gốc tốn 1,63 s; lời gọi mô hình chiếm 1,58 s, tức khoảng 97%. Công cụ truy xuất gần như không tốn thời gian.
3. Chọn một nút để xem nguyên văn đầu vào, đầu ra và gắn phản hồi (feedback) đúng hoặc sai; phần này nằm ngoài ảnh cắt.

Nguồn: LangChain docs, observability quickstart (MIT, langchain-ai/docs)



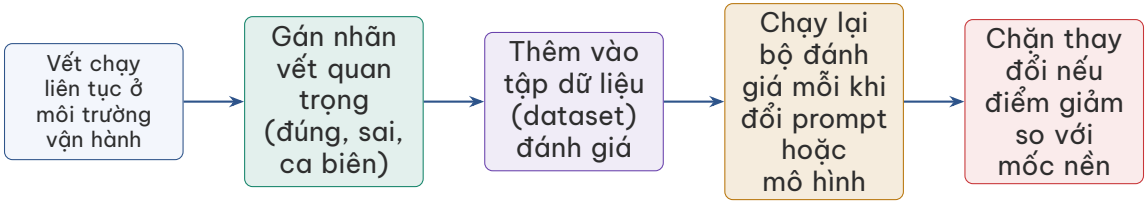
Tags

gen_ai.operation.name	chat
gen_ai.request.model	gpt-4o-mini
gen_ai.response.finish_reasons	▼ ["stop"]
gen_ai.response.id	chatcmpl-Ab1A0Wf1J7UXApTTw5lY7wUcN5Lza
gen_ai.response.model	gpt-4o-mini-2024-07-18
gen_ai.system	openai
gen_ai.usage.input_tokens	15
gen_ai.usage.output_tokens	103

1. Mỗi lời gọi mô hình là một đoạn (span), tên theo quy ước `{operation} {model}`, ví dụ `chat gpt-4o-mini`.
2. Thuộc tính ghi token: `gen_ai.usage.input_tokens` (15) và `gen_ai.usage.output_tokens` (103).
3. Quy ước GenAI vẫn ở mức Development, chưa ổn định. Tên cũ `gen_ai.system` (trong ảnh) được thay bằng `gen_ai.provider.name`.

Nguồn: OpenTelemetry blog 2024 (CC-BY-4.0); semantic-conventions-genai

Từ vết sản xuất đến kiểm thử chống suy giảm



Đánh giá ngoại tuyến (offline)

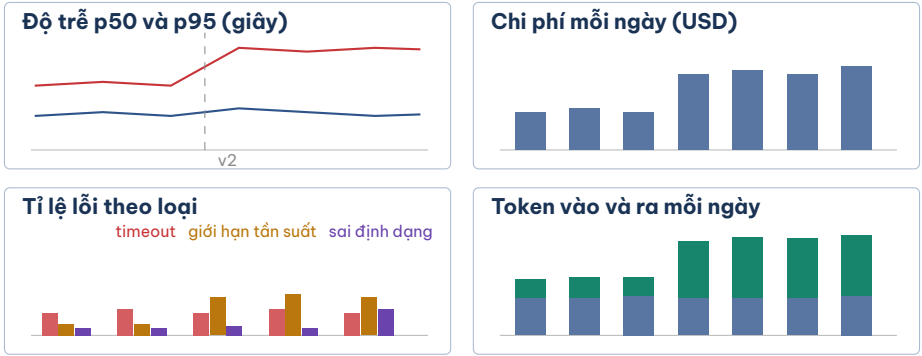
Chạy trước khi triển khai (deployment), trên tập có đáp án chuẩn. Dùng cho kiểm thử chống suy giảm (regression test) và so sánh phiên bản.

Đánh giá trực tuyến (online)

Chạy trên lưu lượng thật, lấy mẫu một tỉ lệ (ví dụ 10%), không cần đáp án chuẩn. Dùng để giám sát (monitoring) chất lượng.

Nguồn: LangSmith docs, evaluation concepts và online evaluations

Bố cục một bảng theo dõi (dashboard) cho ứng dụng LLM



Bảng theo dõi (dashboard) minh hoạ bố cục, không phải số liệu thật. Đường đứt nét đánh dấu thời điểm đổi sang phiên bản prompt mới: ba biểu đồ cùng đổi ở đó.

Quản lý phiên bản prompt

03

Prompt là mã nguồn: bản ghi thay đổi, nhãn, khôi phục và triển khai dần

Có và không có quản lý phiên bản

Không quản lý phiên bản	Có quản lý phiên bản
Prompt sửa trực tiếp, không biết ai đổi gì	Mỗi thay đổi có thông điệp bản ghi (commit message) và người thực hiện
Chi phí tăng nhưng không truy ra nguyên nhân	Mỗi vết gắn mã phiên bản, so sánh được chi phí trước và sau
Độ trễ tăng nhưng không khôi phục được	Khôi phục bằng cách đưa nhãn `production` về bản ghi cũ
Thử nghiệm A/B bị trộn hai phiên bản	Hai phiên bản tách biệt, định tuyến ổn định theo người dùng
Thành viên mới không biết prompt nào đang chạy	Nhãn `production` chỉ rõ phiên bản đang chạy
Lỗi ở môi trường vận hành không tái hiện được	Kéo đúng phiên bản đã chạy tại thời điểm xảy ra lỗi

Nguồn: tổng hợp của giảng viên

Hai sự cố có hồ sơ công khai

xAI · 14/05/2025

Một chỉnh sửa trái phép trong chỉ dẫn hệ thống của Grok khiến chatbot nhấn mạnh một chủ đề chính trị. xAI cho biết quy trình rà soát đã bị vượt qua; sau đó công bố chỉ dẫn hệ thống trên GitHub kèm nhật ký thay đổi và lập đội giám sát 24/7.

Bài học: prompt cần yêu cầu hợp nhất (pull request) và dấu vết kiểm toán.

Nguồn: TechCrunch 15/05/2025 (nguồn thứ cấp)

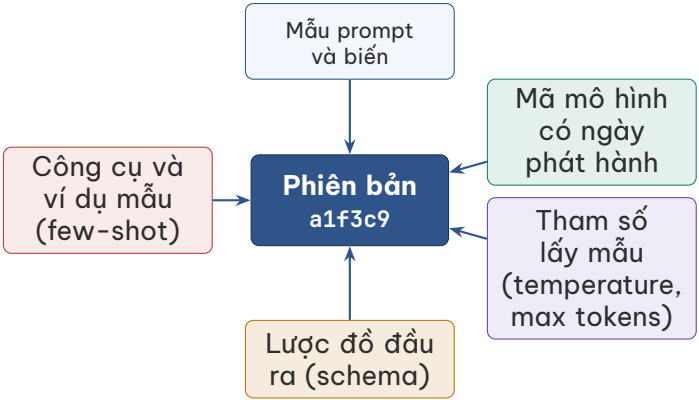
OpenAI · 25 đến 29/04/2025

Bản cập nhật GPT-4o trở nên xu nịnh (sycophancy). Nguyên nhân: tín hiệu thưởng mới từ nút thích và không thích làm yếu tín hiệu thưởng chính. Đánh giá ngoại tuyến và thử nghiệm A/B đều cho kết quả tốt; người kiểm thử chuyên môn thấy hành vi "hơi lệch".

Bài học: suy giảm có thể xảy ra mà không có thay đổi nào trong prompt; đánh giá chỉ bắt được điều nó đo.

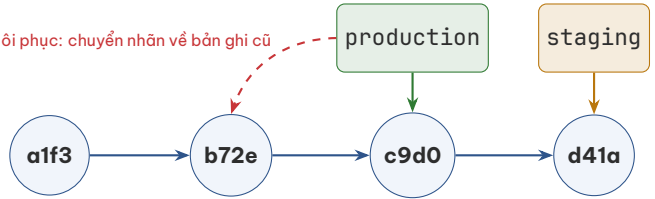
Nguồn: OpenAI, qua Simon Willison, 02/05/2025

Một phiên bản là cả một tổ hợp



- Đổi một thành phần là đổi hành vi; mã phiên bản cần phủ cả tổ hợp.
- Tên mô hình không kèm ngày có thể trỏ tới bản mới hơn: hành vi đổi dù prompt giữ nguyên.
- Ghim mã mô hình có ngày phát hành, ví dụ `gpt-4o-mini-2024-07-18`.

Bản ghi bất biến, nhãn di động



mỗi lần lưu tạo một bản ghi mới, không sửa bản ghi cũ

	LangSmith	Langfuse	MLflow 3
Con trỏ di động	nhãn phiên bản (tag): `staging`, `production` dành riêng	nhãn (label): `production`, `latest`	bí danh (alias): `@production`
Phiên bản	bản ghi có mã băm	số phiên bản tăng dần	bất biến, kiểu Git

Nguồn: tài liệu LangSmith (manage-prompts), Langfuse (prompt management), MLflow (prompt registry), 06/10/2026

Đẩy và kéo prompt bằng LangSmith SDK


```python
from langsmith import Client
from langchain_core.prompts import ChatPromptTemplate

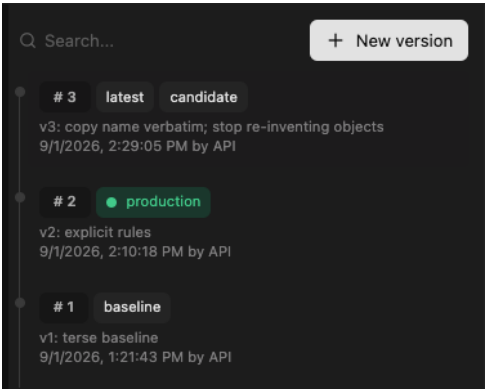
c = Client()
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a support assistant. Answer in {language}."),
    ("human", "{question}"),
])

# PUSH: new commit + message + tag
url = c.push_prompt(
    "support-bot",
    object=prompt,
    commit_tags=["staging"],
    commit_description="feat: add language variable",
)

# PULL: by tag, or by commit hash
p = c.pull_prompt("support-bot:production")
p = c.pull_prompt("support-bot:12344e88")
```

- `langchain.hub` đã bị đánh dấu ngừng dùng (từ langchain 1.0.6, gỡ ở 2.0.0): dùng `langsmith`.
- Môi trường vận hành ghim nhãn `production` hoặc mã băm, không kéo "bản mới nhất".
- Thông điệp bản ghi theo tiền tố `feat`, `fix`, `refactor`.
- Biến có tên (`{language}`) giúp kiểm tra đầu vào trước khi gọi mô hình.
- Webhook kích hoạt mỗi khi có bản ghi mới: nối với CI hoặc đồng bộ sang Git.

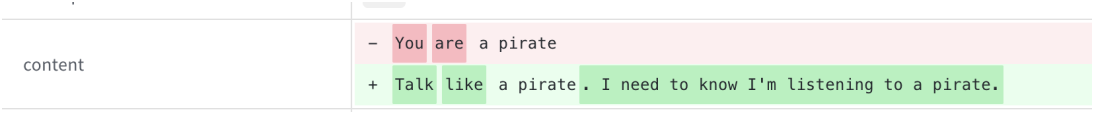
Kho prompt trong Langfuse: phiên bản và nhãn



1. Danh sách ba phiên bản của một prompt, mỗi phiên bản có thông điệp riêng (#1 terse baseline, #2 explicit rules, #3 copy name verbatim).
2. Nhãn `production` chỉ phiên bản đang chạy; nhãn `latest` và `candidate` chỉ bản đang thử.
3. Từ màn hình đầy đủ có thể mở Playground, chạy thử nghiệm và xem các lượt sinh (generation) đã liên kết với phiên bản.

Nguồn: Langfuse docs, prompt management (MIT, langfuse-docs)

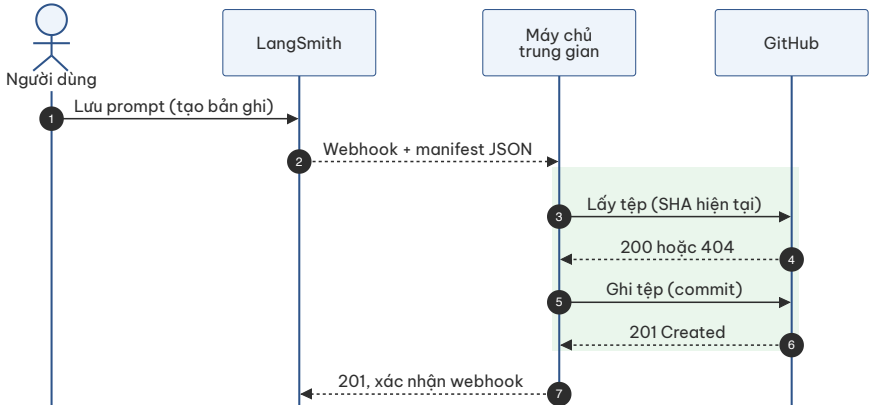
So sánh hai phiên bản prompt như xem một diff



Weave so sánh `pirate_prompt:v0` và `v1`: dòng đỏ là nội dung bị xoá, dòng xanh là nội dung thêm, từng từ đổi được tô đậm. Người rà soát thấy chính xác thay đổi nào đã xảy ra, giống rà soát mã nguồn.

Nguồn: W&B Weave docs, prompt comparison (Apache-2.0, wandb/docs)

Đồng bộ prompt sang Git bằng webhook



Kết quả: có đồng thời kho prompt (giao diện, kiểm thử) và lịch sử Git (diff, rà soát, CI). Vùng xanh là bước máy chủ trung gian ghi tệp vào kho.

Nguồn: LangChain docs, manage prompts (sơ đồ vẽ lại bằng Mermaid)

Prompt dạng tệp YAML trong kho Git

```yaml
# prompts/support-bot.yaml
name: support-bot
version: "1.3.0"
description: "Support prompt with language variable"
model: gpt-4o-mini-2024-07-18
temperature: 0.2
variables: [question, language, context]
template: |
  You are a support assistant. Answer in {language}.
  Use ONLY the context below. If the answer is not
  in the context, say "I don't know".

  Context: {context}
  Question: {question}
```

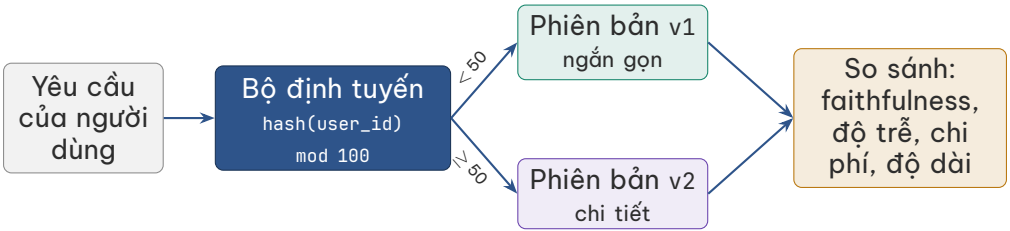
Quy trình

- Mọi thay đổi đi qua `pull request`; diff hiển thị trong GitHub hoặc GitLab.
- CI chạy bộ đánh giá khi tệp prompt đổi.
- Ứng dụng nạp prompt từ tệp thay vì viết cứng trong mã.

Chọn kho prompt hay Git?

- Kho prompt: nhóm lớn, cần giao diện rà soát, người không lập trình sửa được.
- Git: nhóm thiên về lập trình, monorepo, CI chặt chẽ.
- Kho prompt là phụ thuộc lúc chạy: cần bộ nhớ đệm phía máy khách (Langfuse đã có).

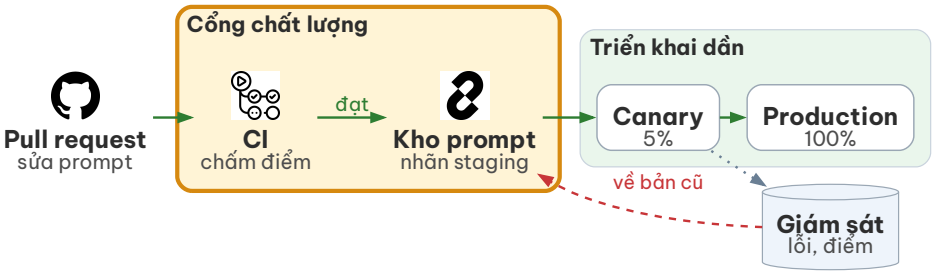
Thử nghiệm A/B hai phiên bản prompt



Băm theo người dùng, không theo yêu cầu: mỗi người luôn gặp một phiên bản trong suốt hội thoại.

Gắn `prompt_version` vào mọi vết để so sánh theo phiên bản. Cỡ mẫu cần thiết: xem phần Đánh giá; 100 truy vấn chỉ là mức khởi động.

Cổng phát hành: từ `pull request` đến môi trường vận hành



Triển khai thử theo tỉ lệ nhỏ (canary): chỉ 5% lưu lượng gặp phiên bản mới trước khi mở rộng lên 100%.

Điều kiện dừng đặt trước: tỉ lệ lỗi, điểm đánh giá trực tuyến, chi phí mỗi yêu cầu. Khôi phục chỉ cần đưa nhãn về bản ghi trước, không cần phát hành lại ứng dụng.

Đánh giá mô hình ngôn ngữ lớn

Vượt ra ngoài accuracy: chỉ số, người chấm là mô hình, cỡ mẫu và cổng CI

Accuracy bỏ sót điều gì

Lỗi	Vì sao accuracy bỏ sót	Chỉ số bắt được
Ảo giác	câu trả lời có thể trùng chuỗi chuẩn mà vẫn thiếu căn cứ, hoặc đúng do may mắn	faithfulness: khẳng định nào không có trong ngữ cảnh (context)
Lạc đề, né tránh	không phải kiểm tra tính đúng; accuracy cần nhãn chuẩn	mức liên quan của câu trả lời (response relevancy)
Bộ truy xuất (retriever) sai	accuracy cuối không cho biết lỗi ở truy xuất (retrieval) hay ở sinh	`context_recall`, `context_precision`
Nhiễu từ đoạn không liên quan	lỗi lan sang câu trả lời mà không lộ nguyên nhân	noise sensitivity
Sai định dạng (không phải JSON)	so khớp văn bản không phân biệt lỗi cấu trúc	kiểm tra xác định: phân tích JSON, đối chiếu lược đồ

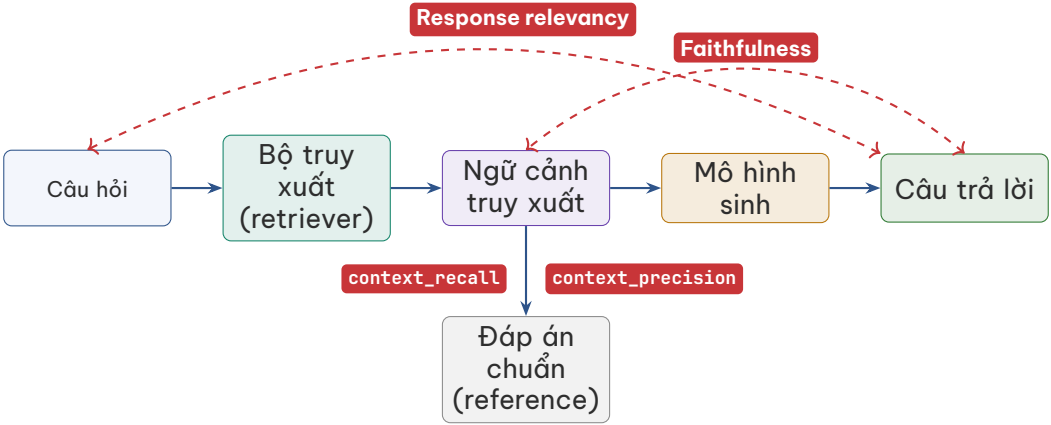
Nguồn: định nghĩa chỉ số từ tài liệu RAGAS; bảng ánh xạ do giảng viên tổng hợp

Ba tầng đánh giá: rẻ đến đắt



Tầng thấp chặn lỗi hiển nhiên trước khi tốn tiền cho tầng cao; tầng cao hiệu chỉnh lại tầng giữa.

RAGAS đo ở đâu trong chuỗi RAG



Trong chuỗi tạo sinh tăng cường truy xuất (RAG), mỗi chỉ số so sánh một cặp thành phần, nhờ đó khoanh vùng lỗi ở truy xuất hay ở sinh.

Các chỉ số RAGAS và cách tính

Chỉ số	Trả lời câu hỏi	Cách tính	Cần đáp án chuẩn
Faithfulness	câu trả lời có bám ngữ cảnh không	số khẳng định được ngữ cảnh hỗ trợ chia tổng số khẳng định	không
Response relevancy	câu trả lời có đúng trọng tâm không	trung bình độ tương đồng cosine giữa câu hỏi và các câu hỏi do mô hình sinh ngược từ câu trả lời (mặc định 3)	không
`context_precision`	đoạn liên quan có xếp hạng cao không	trung bình `Precision@k` có trọng số theo độ liên quan	có
`context_recall`	ngữ cảnh có đủ dữ kiện không	số khẳng định của đáp án chuẩn được ngữ cảnh hỗ trợ chia tổng số khẳng định của đáp án chuẩn	có
Noise sensitivity	câu trả lời sai do đoạn nhiễu	khẳng định sai chia tổng khẳng định; càng thấp càng tốt	có

Ngưỡng như 0,8 chỉ là quy ước kinh nghiệm. Điểm phụ thuộc mô hình chấm và prompt chấm; so sánh chênh lệch giữa các phiên bản trên cùng cấu hình chấm.

Nguồn: docs.ragas.io, mục các chỉ số (06/10/2026); các ngưỡng không có trong tài liệu

RAGAS 0.4: một lần gọi `evaluate()`

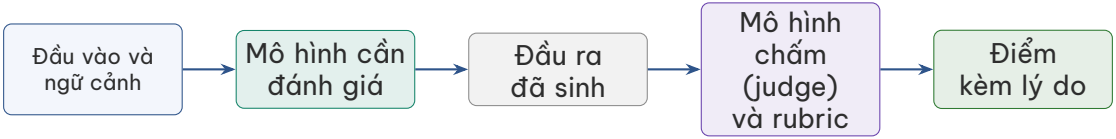
```python
# pip install "ragas==0.4.3" "langchain-community<0.4"
from ragas import evaluate, EvaluationDataset
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from ragas.metrics import (Faithfulness, ResponseRelevancy,
                           LLMContextRecall)  # deprecated path
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

ds = EvaluationDataset.from_list([{
    "user_input": "When was the first Super Bowl?",
    "response": "It was held on Jan 15, 1967.",
    "retrieved_contexts": ["The first AFL-NFL World Championship "
        "Game was played on January 15, 1967."],
    "reference": "January 15, 1967.",
}])
res = evaluate(ds,
    metrics=[Faithfulness(), ResponseRelevancy(),
             LLMContextRecall()],
    llm=LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-mini")),
    embeddings=LangchainEmbeddingsWrapper(OpenAIEmbeddings()))
print(res.to_pandas())
```

Thay đổi API (bản 0.4.3, 13/01/2026)

- `ragas.metrics` vẫn chạy nhưng kèm thông báo ngừng hỗ trợ (deprecation warning); gỡ ở v1.0.
- Mã mới dùng `ragas.metrics.collections` (tên đổi: `AnswerRelevancy`, `ContextPrecision`, `ContextRecall`) và `await metric.ascore(...)`.
- `evaluate()` hiện không nhận chỉ số của `collections`.
- Phải ghim phiên bản: `langchain-community` 0.4.2 làm `import ragas` lỗi.

LLM-as-judge: mô hình chấm điểm đầu ra



Ưu điểm

- Chạy hàng nghìn mẫu mỗi ngày, ổn định, không mệt.
- Không cần đáp án chuẩn nếu rubric rõ (reference-free).
- GPT-4 đồng thuận với người đánh giá chuyên môn 85%, so với 81% giữa người với người (MT-bench, bỏ cặp hoà).

Giới hạn

- Kết quả 85% đo ở một bộ câu hỏi và một thế hệ mô hình, không phải bảo đảm chung.
- Có các thiên lệch hệ thống (trang kế).
- Rubric mơ hồ cho điểm dao động; phải hiệu chỉnh định kỳ với người.

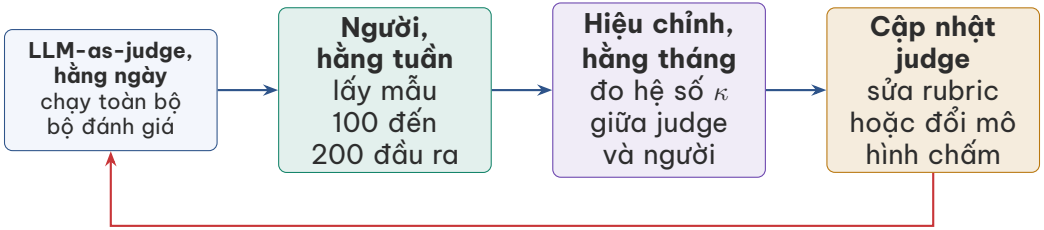
Nguồn: Zheng và cs., Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, NeurIPS 2023 (arXiv 2306.05685)

Thiên lệch của mô hình chấm và cách giảm

Thiên lệch	Biểu hiện đã đo	Cách giảm
Vị trí	đổi thứ tự hai câu trả lời thì kết quả đổi; GPT-4 chỉ nhất quán ở 65% cặp đảo thứ tự	chấm hai lần theo hai thứ tự, chỉ tính thắng khi thắng cả hai
Độ dài	ưa câu trả lời dài hơn dù chất lượng tương đương	rubric nêu rõ độ dài không cộng điểm
Tự ưu ái	mô hình chấm cao đầu ra của chính nó hơn đầu ra của mô hình khác	dùng mô hình chấm khác họ với mô hình được chấm
Toán và lập luận	mô hình chấm lặp lại lỗi của đáp án	chấm có đáp án tham chiếu: tỉ lệ chấm sai giảm từ 70% xuống 15%
Cặp khó	nhiều mô hình mạnh chỉ nhỉnh hơn đoán ngẫu nhiên trên JudgeBench	hiệu chỉnh với người; không tuyên bố "đã đủ tin cậy"

Nguồn: Zheng 2023 (2306.05685); Saito 2023 (2310.10076); Panickssery 2024 (2404.13076); Wang 2023 (2305.17926); JudgeBench (2410.12784)

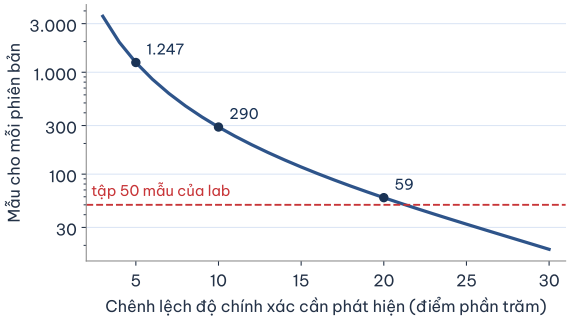
Vòng đánh giá kết hợp và độ đồng thuận



Đọc hệ số Cohen κ: 0,61 đến 0,80 thường gọi là đồng thuận đáng kể (Landis và Koch, 1977). Đây là quy ước tham khảo, không có chứng cứ thực nghiệm, và κ phụ thuộc tỉ lệ các nhãn.

Công cụ gán nhãn: Label Studio (mã nguồn mở), Argilla, Scale AI, Prolific. Ngưỡng chọn (ví dụ $\kappa < 0{,}7$ thì sửa rubric) do từng nhóm đặt.

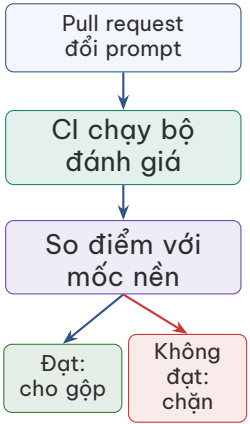
Cần bao nhiêu mẫu để so sánh hai prompt



1. Hai tỉ lệ độc lập, $\alpha = 0{,}05$, lực kiểm định 80%:
   $n = (z_{\alpha/2}+z_{\beta})^2\,[p_1q_1+p_2q_2]\,/\,(p_1-p_2)^2$,
   với $q = 1-p$.
2. Từ mốc 70%: cải thiện 5 điểm cần khoảng 1.250 mẫu mỗi phiên bản; 10 điểm cần 290; 20 điểm cần 59.
3. Tập 50 mẫu của lab chỉ đủ cho chênh lệch cỡ 20 điểm: dùng để kiểm tra nhanh, chưa đủ để kết luận.
4. Thiết kế ghép cặp (cùng câu hỏi cho hai prompt, kiểm định McNemar): khoảng 312 đến 940 mẫu cho 5 điểm.

Nguồn: tính toán của giảng viên (xấp xỉ chuẩn), không phải trích dẫn

Cổng đánh giá trong CI




```python
# tests/test_prompt_regression.py
from statistics import mean

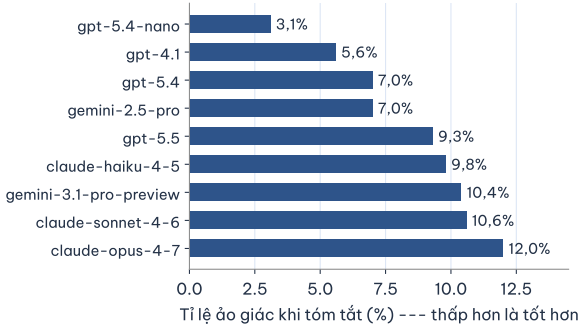
TOLERANCE = 0.02

def test_no_regression(candidate_scores, baseline_scores):
    assert mean(candidate_scores) >= \
        mean(baseline_scores) - TOLERANCE

def test_format_always_valid(outputs):
    assert all(is_valid_json(o) for o in outputs)
```

Kiểm tra xác định đặt trước, chấm bằng LLM đặt sau. Công cụ có sẵn: Promptfoo (`npx promptfoo@latest eval --fail-on-error`) và DeepEval (`assert_test` với `threshold`).

Bảng xếp hạng công khai nói gì và không nói gì



1. Chỉ số: tỉ lệ ảo giác khi tóm tắt tài liệu, chấm bằng HHEM-2.3; không phải faithfulness của RAGAS.
2. Mô hình nhỏ đứng đầu bảng; mô hình lớn có tỉ lệ cao hơn. Lớn hơn không đồng nghĩa ít ảo giác hơn.
3. Bảng faithfulness của GPT, Claude, Gemini trong bản cũ đã bỏ: không có nguồn công khai.
4. Kết luận: đo trên dữ liệu của chính hệ thống.

Nguồn: Vectara Hallucination Leaderboard, README cập nhật 22/09/2026 (HHEM-2.3)

W&B Weave và công cụ đánh giá

05

Tự động ghi vết, so sánh phiên bản và chọn công cụ

Weave: ghi vết và `Evaluation`

```python
import asyncio, weave
from weave import Evaluation, Model

weave.init("day22-demo")            # needs W&B login
examples = [{"question": "What is the capital of France?",
             "expected": "Paris"}]

@weave.op()
def match_score(expected: str, output: dict) -> dict:
    return {"match": expected == output["generated_text"]}

class MyModel(Model):
    prompt: str                     # versioned with the model
    @weave.op()
    def predict(self, question: str):
        return {"generated_text": "Paris"}

asyncio.run(Evaluation(dataset=examples, scorers=[match_score])
            .evaluate(MyModel(prompt="v1")))
```

- Mỗi hàm gắn `@weave.op()` được ghi đầu vào, đầu ra và độ trễ.
- `Model` chứa cả prompt: đổi thuộc tính là sinh phiên bản mới.
- Hàm chấm điểm (scorer) bắt buộc có đối số tên `output`.
- Các lần chạy đánh giá được so sánh cạnh nhau trong giao diện.
- W&B thuộc CoreWeave từ 05/05/2025; tài liệu Weave nay ở `docs.coreweave.com`.

Chọn công cụ đánh giá

Công cụ	Giấy phép	Điểm dùng
LangSmith	SDK MIT; nền tảng thương mại	`evaluate()` trên tập dữ liệu; đánh giá ngoại tuyến và trực tuyến
W&B Weave	Apache-2.0	`Evaluation` và scorer, ghi vết, so sánh phiên bản
DeepEval	Apache-2.0	kiểm thử kiểu pytest với `assert_test` và ngưỡng
Promptfoo	MIT	CLI và cấu hình YAML, cổng CI, kiểm thử tấn công giả lập (red teaming); nay thuộc OpenAI
Inspect AI	MIT	khung của UK AI Security Institute, hơn 200 bộ đánh giá dựng sẵn
RAGAS	Apache-2.0	chỉ số riêng cho RAG
OpenAI Evals	MIT	khung và kho bộ đánh giá; README nay hướng dẫn chạy trong bảng điều khiển của OpenAI

Nguồn: GitHub và PyPI của từng dự án, 06/10/2026

Rào chắn an toàn và giám sát

Phòng thủ nhiều lớp cho ứng dụng LLM

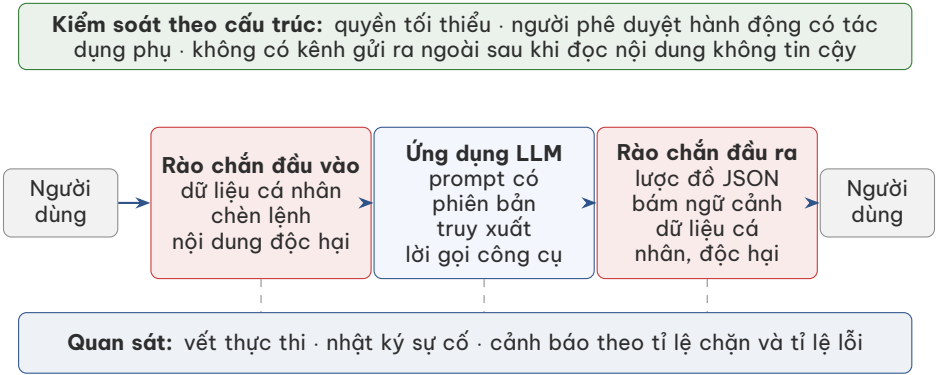
OWASP Top 10 cho ứng dụng LLM, bản 2025

01 **Chèn lệnh (prompt injection)**
02 **Lộ thông tin nhạy cảm**
03 Chuỗi cung ứng
04 Đầu độc dữ liệu và mô hình
05 **Xử lý đầu ra không an toàn**
06 Quyền hạn quá mức (excessive agency)
07 Lộ chỉ dẫn hệ thống
08 Điểm yếu của vector biểu diễn (embedding)
09 **Thông tin sai lệch**
10 Tiêu thụ tài nguyên không giới hạn

Mục màu đỏ là các mục mà rào chắn ở đầu vào và đầu ra xử lý trực tiếp trong bài này. OWASP chưa công bố bản LLM Top 10 mới hơn 2025; riêng danh sách "Top 10 for Agentic Applications for 2026" ra ngày 09/12/2025.

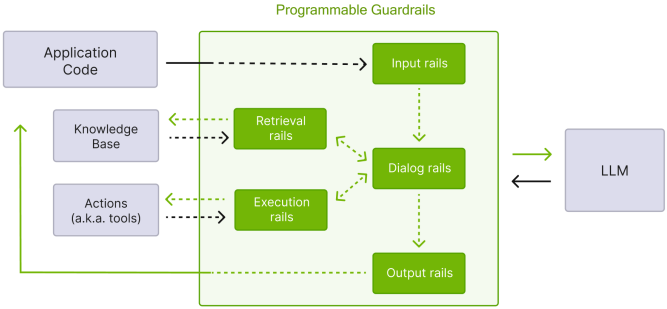
Nguồn: genai.owasp.org/llm-top-10 (06/10/2026)

Phòng thủ nhiều lớp



Mỗi lớp bắt một loại lỗi khác nhau; không lớp nào đứng một mình.

NeMo Guardrails: năm loại rào chắn



High-level flow through programmable guardrails.

1. `input rails` kiểm tra hoặc biến đổi đầu vào trước khi tới mô hình.
2. `dialog rails` điều hướng luồng hội thoại.
3. `retrieval rails` lọc các đoạn truy xuất; `execution rails` kiểm soát lời gọi công cụ.
4. `output rails` kiểm tra đầu ra trước khi trả về.

Phiên bản 0.24.1, giấy phép Apache-2.0.

Nguồn: NVIDIA NeMo Guardrails (Apache-2.0, © NVIDIA)

Bộ phân loại an toàn: nội dung độc hại và chèn lệnh

Công cụ	Việc làm	Giấy phép
Llama Guard 4 (12B)	phân loại nội dung độc hại, 14 nhóm S1 đến S14, đa phương thức	Llama 4 Community
Prompt Guard 2	phát hiện chèn lệnh và vượt giới hạn; 86M tham số 92,4 ms, 22M tham số 19,3 ms (A100, 512 token)	Llama 4 Community
NeMo Guardrails	khung năm loại rào chắn, ngôn ngữ Colang	Apache-2.0
Azure Prompt Shields	quét tấn công từ lời nhắc người dùng và từ tài liệu	dịch vụ thương mại
omni-moderation	13 nhóm, văn bản và hình ảnh	miễn phí qua API
gpt-oss-safeguard	phân loại theo chính sách do nhà phát triển viết lúc suy luận	Apache-2.0
Qwen3Guard	119 ngôn ngữ, ba mức nghiêm trọng	Apache-2.0 (Gen-8B)

Nguồn: thẻ mô tả mô hình (model card) và tài liệu của từng dự án (06/10/2026); độ trễ Prompt Guard theo LlamaFirewall, arXiv 2505.03574

Guardrails AI 0.11: `Guard().use(...)` nhận đối tượng validator

```python
# pip install guardrails-ai guardrails-ai-valid-json
from guardrails import Guard, OnFailAction
from guardrails.validators import (
    Validator, register_validator, PassResult, FailResult)
from guardrails_ai.valid_json import ValidJson

@register_validator(name="demo/no_secret", data_type="string")
class NoSecret(Validator):
    def _validate(self, value, metadata=None):
        if "sk-" in value:
            return FailResult(error_message="secret",
                fix_value=value.replace("sk-", "***"))
        return PassResult()

guard_json = Guard().use(ValidJson(on_fail=OnFailAction.EXCEPTION))
guard_json.validate('{"a": 1}').validation_passed  # True
guard_secret = Guard().use(NoSecret(on_fail=OnFailAction.FIX))
guard_secret.validate("key sk-123").validated_output
# -> 'key ***123'
```

Thay đổi API (0.11.0, 14/08/2026)

- `use()` nhận đối tượng (instance) của validator; `on_fail` đặt ở validator.
- Không có `use_many`; kiểu cũ `guard(llm_api=...)` không còn trong tài liệu hiện hành.
- Validator từ Hub cài bằng pip (`guardrails-ai-detect-pii`); chưa kiểm chạy.
- Tài liệu vẫn hiện cú pháp cũ: thử lại trước khi dùng.

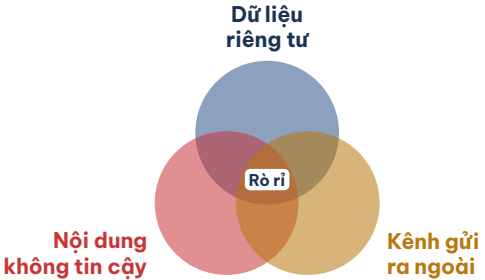
Sáu hành động khi validator thất bại

on_fail	Hành vi quan sát được	Dùng khi
exception	ném ValidationError	luồng phải dừng
fix	thay đầu ra bằng giá trị đã sửa, kết quả hợp lệ	che dữ liệu cá nhân, thay khoá
filter	đầu ra rỗng (None); tài liệu: chỉ lọc trường lỗi	bỏ trường hỏng
refrain	không trả đầu ra (None)	nội dung không an toàn
noop	giữ nguyên đầu ra, ghi thất bại vào nhật ký	giai đoạn đo, chưa chặn
reask, fix_reask	gọi lại mô hình kèm thông báo lỗi (chưa kiểm chạy); fix_reask sửa xác định trước rồi mới hỏi lại	JSON sai định dạng

Nguồn: tài liệu validator_ on_ fail_ actions; hành vi quan sát khi chạy Guardrails AI 0.11.0 ngày 06/10/2026

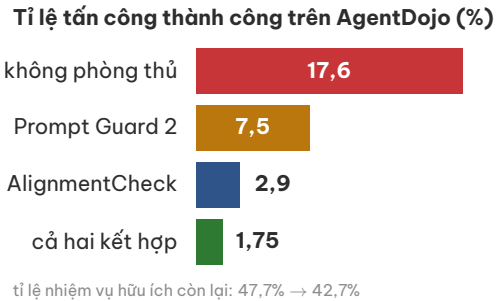
Chèn lệnh: trực tiếp, gián tiếp và lethal trifecta

- **Trực tiếp:** chính người dùng viết lệnh để đổi hành vi mô hình.
- **Gián tiếp:** văn bản độc hại nằm trong email, trang web hoặc tệp mà hệ thống đọc.
- OWASP nêu chưa có biện pháp ngăn chặn tuyệt đối; RAG và tinh chỉnh (fine-tuning) không giải quyết triệt để.
- Ba điều kiện cùng lúc gây rò rỉ dữ liệu (lethal trifecta) là ba vòng bên phải; cách chắc chắn là loại bỏ ít nhất một vòng bằng thiết kế.



Nguồn: OWASP LLM01:2025; Simon Willison, The lethal trifecta, 16/06/2025

Bộ lọc giảm rủi ro nhưng không chặn hết



- Meta tự chạy phép đo trên một phần hẹp của AgentDojo: không đại diện cho mọi cuộc tấn công.
- Rủi ro còn lại lớn hơn 0 và tiện ích giảm khi thêm lớp phòng thủ.
- Bộ lọc là lớp xác suất; ranh giới bảo mật phải đến từ thiết kế.
- CaMeL tách luồng điều khiển khỏi dữ liệu không tin cậy: giải 77% nhiệm vụ với bảo đảm có thể chứng minh, so với 84% khi không phòng thủ.

Nguồn: LlamaFirewall (arXiv 2505.03574); AgentDojo (arXiv 2406.13352); CaMeL (arXiv 2503.18813)

Hai sự cố có hồ sơ công khai

EchoLeak · CVE-2025-32711

Lỗ hổng chèn lệnh không cần thao tác của người dùng trong Microsoft 365 Copilot: một email được thiết kế sẵn đủ để rò rỉ dữ liệu từ xa. Chuỗi tấn công vượt qua bộ phân loại XPIA của Microsoft, cơ chế che liên kết và chính sách CSP.

Bài học: bộ phân loại không thay cho thiết kế kiến trúc.

Nguồn: arXiv 2509.10540

Moffatt kiện Air Canada · 2024

Chatbot của hãng cung cấp thông tin sai cho khách hàng. Cơ quan phân xử dân sự của British Columbia (Canada) bác lập luận rằng chatbot là một chủ thể pháp lý riêng; doanh nghiệp chịu trách nhiệm về thông tin trên website của mình, dù thông tin đó do chatbot đưa ra.

Bài học: đây là lỗi thông tin sai lệch (OWASP LLM09), không phải cuộc tấn công; doanh nghiệp sở hữu đầu ra.

Nguồn: 2024 BCCRT 149, qua McCarthy Tétrault

Dữ liệu cá nhân và nhật ký sự cố

Phát hiện dữ liệu cá nhân (PII)

- Regex và kiểm tra tổng kiểm: chính xác, nhanh, hợp với mã có cấu trúc (email, số thẻ, số căn cước 12 chữ số).
- Nhận dạng thực thể (NER): bắt tên và địa chỉ tự do, nhưng nhiễu hơn và phụ thuộc ngôn ngữ.
- Presidio kết hợp cả hai; dự án đang chuyển sang tổ chức cộng đồng và nêu rõ không bảo đảm phát hiện hết.

Trường nên có trong nhật ký sự cố

- mã yêu cầu và vết; phiên bản prompt và mã mô hình
- tên và phiên bản rào chắn; giai đoạn (đầu vào, đầu ra, công cụ)
- kết luận, điểm số, hành động `on_fail`
- mã nhóm OWASP; đoạn trích đã che hoặc mã băm, không lưu dữ liệu gốc
- độ trễ của rào chắn; kết quả rà soát của người

Cảnh báo: ngưỡng minh hoạ và cách đặt ngưỡng

Điều kiện (minh hoạ)	Hành động
độc hại lớn hơn 0,5%	báo Slack, rà soát thủ công
ảo giác lớn hơn 5%	gọi trực ca, điều tra
phát hiện rò rỉ dữ liệu cá nhân	chặn ngay, ghi nhật ký
hỏi lại (reask) quá 3 lần mỗi giờ	xem lại chất lượng prompt
rào chắn thất bại lớn hơn 2%	kiểm tra cấu hình

Các ngưỡng này không có nguồn công bố: hiệu chỉnh theo mốc nền của từng hệ thống và từng cách đo.

Nguồn: Google SRE Workbook, Alerting on SLOs (sre.google/workbook/alerting-on-slos)

Cách đặt có cơ sở: đốt ngân sách lỗi (burn rate)

- Định nghĩa một chỉ số dịch vụ, ví dụ tỉ lệ phản hồi qua rào chắn.
- Với mục tiêu 99,9%, SRE Workbook gợi ý: gọi trực khi tốc độ đốt 14,4 lần trong 1 giờ (cửa sổ ngắn 5 phút); 6 lần trong 6 giờ; 1 lần trong 3 ngày thì mở phiếu.

Trình diễn và thực hành

07

Các phần trình diễn (demo) và bốn nhiệm vụ lab:
LangSmith, A/B prompt, RAGAS, Guardrails AI

Chương trình trình diễn (40 phút)

LIVE DEMO

1. **RAG và LangSmith (10 phút):** gắn `@traceable`, xem cây vết, tìm bước nghẽn, lọc vết chậm.
2. **A/B prompt (10 phút):** đẩy hai phiên bản lên kho prompt, định tuyến 50/50 theo người dùng, gắn `prompt_version`, so sánh sau 100 truy vấn.
3. **Báo cáo RAGAS (8 phút):** 50 cặp hỏi đáp, chạy cả hai phiên bản, so faithfulness và mức liên quan.
4. **Guardrails AI (8 phút):** đưa vào dữ liệu cá nhân và JSON hỏng, quan sát `fix` và `reask`, ghi nhật ký sự cố.
5. **Tham quan bảng theo dõi (4 phút):** độ trễ, chi phí theo ngày, lỗi theo loại, token.

Lab 22: nhiệm vụ 1 và 2

1. Ghi vết LangSmith

1. `pip install -U langsmith langchain-openai`
2. đặt `LANGSMITH_TRACING=true` và `LANGSMITH_API_KEY`
3. dựng RAG đơn giản: bộ truy xuất và LLM
4. gắn `@traceable` cho hàm tự viết
5. chạy trên 100 truy vấn, kiểm tra vết trên bảng theo dõi

2. Phiên bản prompt và A/B

1. đẩy hai phiên bản lên kho prompt: v1 ngắn (tối đa 100 từ), v2 chi tiết
2. định tuyến theo `hash(user_id) mod 2`
3. gắn `prompt_version` vào siêu dữ liệu của vết
4. so sánh độ trễ, chi phí, độ dài đầu ra giữa hai phiên bản
5. ghi cỡ mẫu và nêu giới hạn của kết luận

3. Đánh giá bằng RAGAS

1. tạo 50 cặp hỏi đáp từ tài liệu miền
2. `pip install "ragas==0.4.3" "langchain-openai<1" "langchain-community<0.4"`
3. chạy `Faithfulness`, `ResponseRelevancy`, `LLMContextPrecisionWithReference`
4. so hai phiên bản prompt trên cùng cấu hình chấm
5. báo cáo phiên bản nào tốt hơn, kèm lý do và cỡ mẫu; mục tiêu faithfulness lớn hơn 0,8 chỉ là quy ước tham khảo

4. Rào chắn Guardrails AI

1. `pip install guardrails-ai guardrails-ai-valid-json`
2. dựng `Guard` với `ValidJson` và một validator che dữ liệu cá nhân
3. thử đầu vào có email và số điện thoại, kiểm tra việc che
4. thử đầu ra không phải JSON, kiểm tra hành vi `on_fail`
5. ghi sự cố vào tệp để làm dấu vết kiểm toán

Lỗi cài đặt thường gặp trong lab

- ☑ Dùng biến `LANGSMITH_*`, không phải `LANGCHAIN_*`; gói miễn phí giữ vết cơ bản 14 ngày, xuất bằng chứng trước hạn
- ☑ `langchain.hub` đã ngừng dùng: kéo và đẩy prompt bằng `langsmith.Client`
- ☑ Ghim `langchain-community<0.4` khi cài RAGAS; `import ragas` lỗi với bản 0.4.2
- ☑ Guardrails AI: truyền đối tượng validator vào `use()`, đặt `on_fail` ở validator
- ☐ Không đưa khoá API vào mã nguồn hoặc bản chụp màn hình nộp bài

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Prompt là mã nguồn: phiên bản là tổ hợp prompt, mô hình và tham số; môi trường vận hành ghim nhãn hoặc mã băm; khôi phục bằng cách chuyển nhãn.
2. Ghi vết mọi lời gọi LLM bằng định dạng mở (OpenTelemetry); vết là nguồn của chi phí, độ trễ và tập dữ liệu đánh giá.
3. Đánh giá theo tầng: kiểm tra xác định, rồi LLM-as-judge đã hiệu chỉnh với người; chọn cỡ mẫu theo chênh lệch cần phát hiện.
4. Phòng thủ nhiều lớp: bộ lọc giảm rủi ro, thiết kế theo cấu trúc mới tạo ranh giới an toàn.

Bài tiếp theo

Khôi phục thảm hoạ và tính sẵn sàng cao cho hạ tầng AI

"Vùng us-east-1 ngừng hoạt động và điểm cuối (endpoint) phục vụ mô hình nằm ở đó. Hệ thống có bao nhiêu phút trước khi khách hàng nhận ra?"

Bài tập về nhà

- Hoàn thành Lab 22: nộp bản chụp màn hình vết, lịch sử phiên bản prompt, báo cáo RAGAS và nhật ký sự cố.
- Đọc trước: thời gian khôi phục mục tiêu (RTO) và điểm khôi phục mục tiêu (RPO).
- Mở rộng: so sánh hai mô hình vector biểu diễn (embedding) bằng `context_precision` của RAGAS.

Tài liệu tham khảo

1. LangChain, *LangSmith docs*: tracing, manage prompts, evaluation. `docs.langchain.com/langsmith`
2. OpenTelemetry, *GenAI semantic conventions*. `github.com/open-telemetry/semantic-conventions-genai`
3. RAGAS, *Metrics*. `docs.ragas.io`
4. Zheng và cs., *Judging LLM-as-a-Judge*, NeurIPS 2023. `arxiv.org/abs/2306.05685`
5. OWASP, *Top 10 for LLM Applications 2025*. `genai.owasp.org/llm-top-10`
6. Guardrails AI. `github.com/guardrails-ai/guardrails`; NVIDIA NeMo Guardrails. `github.com/NVIDIA-NeMo/Guardrails`
7. Chi và cs., *LlamaFirewall*. `arxiv.org/abs/2505.03574`; Debenedetti và cs., *AgentDojo*. `arxiv.org/abs/2406.13352`
8. Vectara, *Hallucination Leaderboard*. `github.com/vectara/hallucination-leaderboard`
9. Google, *SRE Workbook: Alerting on SLOs*. `sre.google/workbook/alerting-on-slos`

Hỏi & Đáp



Cảm ơn!

Slide, mẫu lab và tài nguyên: lms.vinuni.edu.vn